

Spring AOP: Implementing Advice Types

Spring AOP allows us to execute additional logic before, after Spring bean methods without mixing that logic into the business logic.

Typical cross-cutting concerns include:

- Logging
- Authorization
- Auditing
- Performance measurement
- Metrics
- Technical validation
- Context management

In this chapter, we will create a basic aspect and implement the five principal advice types provided by Spring AOP:

1. `@Before`
 2. `@AfterReturning`
 3. `@AfterThrowing`
 4. `@After`
 5. `@Around`
-

1. Add the AOP Dependency

The dependency name differs between Spring Boot 3 and Spring Boot 4.

Spring Boot 3.x

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Spring Boot 4.x

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aspectj</artifactId>
</dependency>
```

Once the appropriate starter is present, Spring Boot configures the required AOP infrastructure automatically. In a normal Spring Boot application, we do not need to manually add `@EnableAspectJAutoProxy`.

2. Recommended Package Structure

```
com. [redacted] .studentmanagement
|
├── StudentManagementApplication
├── controller
|   └── StudentController
├── service
|   └── StudentService
├── repository
|   └── StudentRepository
└── aspect
    └── LoggingAspect
```

The aspect class should normally be placed inside the main application package or one of its subpackages. This allows Spring Boot's component scanning to discover it automatically.

Creating the First Aspect

3. Create the Target Bean

A **target bean** is the ordinary Spring bean whose method we

```
package com. .studentmanagement.servic

import org.springframework.stereotype.Service;

@Service
public class StudentService {

    public String createStudent() {
        System.out.println("Actual createStudent() method executed");
        return "Student created successfully";
    }

    public String getStudent() {
        System.out.println("Actual getStudent() method executed");
        return "Student details";
    }
}
```

Here:

- `StudentService` is the target class.
- The Spring-managed `StudentService` instance is the target object.
- `createStudent()` and `getStudent()` are possible target methods.

4. Create the Aspect Class

```
package com. .studentmanagement.aspect;

import org.aspectj.lang.annotation.Aspect;
```

```

import org.aspectj.lang.annotation.Before;
import org.springframework.stereotype.Component;

@Aspect
@Component
public class LoggingAspect {

    @Before(
        "execution(String " +
        "com. .... .studentmanagement.service.StudentService" +
        ".createStudent())"
    )
    public void logBeforeMethodExecution() {
        System.out.println(
            "A StudentService method is about to execute"
        );
    }
}

```

This class contains three important annotations.

@Aspect

Declares that the class contains AOP configuration, such as pointcuts and advice.

It tells Spring:

This class describes cross-cutting behaviour that must be applied to selected method executions.

@Component

Registers the aspect as a Spring bean.

@Aspect describes the role of the class, but it does not by itself register an object in the Spring container. **@Component** makes the class discoverable through component scanning.

@Before

Declares that the advice method must execute before every its pointcut expression.

5. Understanding the Pointcut Expressio

```
execution(  
    String  
    com. .... .studentmanagement.service.StudentService  
    .createStudent  
    ()  
)
```

The expression can be read from left to right:

Part	Meaning
execution	Match a method execution
String	The method must return String
StudentService	The method must belong to this class
createStudent	The method name must be createStudent
()	The method must take no arguments

The complete string inside @Before is a **pointcut expression**.

```
"execution(String com. .... .studentmanagement.service.Stu  
dentService.createStudent())"
```

The method containing the additional behaviour is the **advice**.

```
@Before("...")  
public void logBeforeMethodExecution() {  
    System.out.println(  
        "A StudentService method is about to execute"  
    )  
}
```

```
);  
}
```

The matched business method is the **target method**.

```
public String createStudent() {  
    return "Student created successfully";  
}
```

Target Object and Proxy Object

6. How Does the Advice Execute Automatically?

The `StudentService` class does not explicitly call `LoggingAspect`. Still, the advice executes before the service method.

Spring makes this possible by placing a **proxy object** in front of the target object.

Conceptually, the proxy behaves like a wrapper:

```
public String createStudent() {  
    loggingAspect.logBeforeMethodExecution();  
    return target.createStudent();  
}
```

The real proxy created by Spring is generic and considerably more sophisticated, but this is the correct mental model.

7. Target Object

The target object contains the actual business implementation.

```
public class StudentService {  
  
    public String createStudent() {  
        return "Student created";  
    }  
}
```

```
}  
}
```

Conceptually:

```
Target object = actual StudentService instance
```

8. Proxy Object

The proxy is the Spring-generated object that stands in front of the target object and intercepts matching method calls.

```
Caller → Proxy → Advice chain → Target object
```

The proxy can run additional behaviour before or after delegating the call to the target.

Are the target and proxy two different objects?

Conceptually, yes. Spring may maintain:

- A proxy object
- A target object behind that proxy

However, they should not be imagined as two ordinary, independently injectable application beans.

The normal bean name exposes the proxy:

```
studentService → proxy object
```

The target is held behind the proxy. The Spring container manages the advised service as one logical bean, while callers normally interact with the proxy.

Advice Types in Spring AOP

9. What Is an Advice?

An advice answers one fundamental question:

What additional logic should run before, after, or around a execution?

The general flow can be represented as:

```
Caller
↓
Advice logic
↓
Target method
↓
Advice logic
↓
Result or exception
```

Spring provides five principal advice types:

- `@Before`
- `@AfterReturning`
- `@AfterThrowing`
- `@After`
- `@Around`

10. The `try-catch-finally` Mental Model

The advice types can be understood by comparing them with ordinary Java control flow.

```
beforeLogic();

try {
    Object result = targetMethod();

    afterReturningLogic(result);
}
```



```

        return result;

    } catch (Throwable exception) {

        afterThrowingLogic(exception);
        throw exception;

    } finally {

        afterLogic();

    }

```

Approximate mapping:

Java control-flow position	Spring advice
Before entering <code>try</code>	<code>@Before</code>
After a successful return	<code>@AfterReturning</code>
Inside <code>catch</code>	<code>@AfterThrowing</code>
Inside <code>finally</code>	<code>@After</code>
The entire surrounding block	<code>@Around</code>

`@Around` is more powerful than simply combining the other four. It controls whether, when, and how the remaining invocation chain proceeds.

`@Before` Advice

11. What Does `@Before` Do?

`@Before` executes before the matched target method begins.

```

@Before(
    "execution(* " +
    "com. .studentmanagement.service.PaymentService.*"
    " (..) )"
)

```

```

public void logMethodEntry(JoinPoint joinPoint

    System.out.println(
        "Entering method: " +
        joinPoint.getSignature().getName()
    );

    System.out.println(
        "Arguments: " +
        Arrays.toString(joinPoint.getArgs())
    );
}

```

Common uses include:

- Logging method entry
- Reading method arguments
- Checking authorization
- Checking technical preconditions
- Populating request or tracing context
- Validating a common technical policy

12. Using **JoinPoint**

JoinPoint represents the current intercepted method execution.

It allows the advice to inspect contextual information such as:

```

joinPoint.getSignature();
joinPoint.getSignature().getName();
joinPoint.getArgs();
joinPoint.getTarget();
joinPoint.getThis();

```

Common methods

Method	Information returned
<code>getSignature()</code>	Method signature information
<code>getArgs()</code>	Arguments passed to the target method
<code>getTarget()</code>	The underlying target object
<code>getThis()</code>	The proxy object currently handling the call

`JoinPoint` allows inspection, but it does not allow the advice to continue, skip, or repeat the target invocation.

13. Can `@Before` Prevent the Target Method?

`@Before` does not have a `doNotProceed()` operation because it does not directly control the invocation.

However, the advice can throw an exception:

```
@Before(
    "execution(* " +
    "com.████████.studentmanagement.service.PaymentService"
    +
    ".makePayment(..))"
)
public void checkAccess() {

    boolean allowed = false;

    if (!allowed) {
        throw new SecurityException(
            "You cannot make this payment"
        );
    }
}
```

Therefore:

- ## 14. Can @Before Modify Method Arguments?

```
Object[] arguments = joinPoint.getArgs();
```

For argument replacement, use `@Around`:

```
joinPoint.proceed(modifiedArguments);
```

@AfterReturning Advice

15. When Does `@AfterReturning` Execute?

```
@AfterReturning(
    "execution(* * +
    \"com. ....studentmanagement.service.PaymentService.*
    (..)\"
)
public void logSuccessfulExecution() {
    System.out.println(
```

```

        "PaymentService method completed succe
    );
}

```

Successful execution

```

Target method
↓
Returns normally
↓
@AfterReturning executes

```

Exceptional execution

```

Target method
↓
Throws an exception
↓
@AfterReturning does not execute

```

16. Accessing the Returned Value

Use the `returning` attribute to bind the returned value to an advice parameter.

```

@AfterReturning(
    pointcut =
        "execution(* " +
        "com. .studentmanagement.service.PaymentServi
ce.*(..))",
    returning = "result"
)
public void logReturnedValue(String result) {
    System.out.println(
        "Method returned: " + result
    )
}

```

```
);  
}
```

The value of `returning` must match the parameter name:

```
returning = "result"
```

```
public void logReturnedValue(String result)
```

The parameter type also affects matching.

```
public void logReturnedValue(String result)
```

This applies only when the returned value is compatible with `String`.

For a generic advice, use `Object`:

```
public void logReturnedValue(Object result)
```

17. Can `@AfterReturning` Replace the Returned Value?

No.

```
@AfterReturning(  
    pointcut = "...",  
    returning = "result"  
)  
public Object replaceResult(Object result) {  
    return "Different result";  
}
```

The value returned by the advice method is ignored. It does not replace the value returned by the target method.

Therefore, an `@AfterReturning` method should normally return `void`:

```
public void logResult(Object result)
```

Use `@Around` when the caller must receive a changed, wrapped different result.

18. What About Mutating the Returned Object

Suppose the target returns a mutable object:

```
PaymentReceipt receipt
```

The advice receives the same object reference and could technically modify it:

```
receipt.setStatus("CHANGED BY ASPECT");
```

This is normally poor design because it hides meaningful data transformation outside the visible business flow.

Use `@AfterReturning` mainly for observation:

- Logging a returned value
- Recording a metric
- Writing an audit record
- Publishing a success-related event

Use `@Around` when deliberate result transformation is genuinely required.

19. Normal Completion Means How the Method Exits

Consider a target method that catches an internal exception:

```
public String makePayment(double amount) {  
  
    try {  
        throw new RuntimeException("Gateway failed");  
    } catch (RuntimeException exception) {
```

```

        return "Fallback response";
    }
}

```

An exception occurred inside the method, but it did not escape and ultimately returned a value.

Therefore:

```

@AfterReturning executes
@AfterThrowing does not execute

```

Advice reacts to how the target method exits, not to every internal event that occurs inside it.

@AfterThrowing Advice

20. When Does @AfterThrowing Execute?

`@AfterThrowing` runs when the matched target method exits by throwing an exception.

```

@AfterThrowing(
    "execution(* * +
    "com. ....studentmanagement.service.PaymentService.*
    (..))"
)
public void logFailure() {
    System.out.println(
        "PaymentService method failed"
    );
}

```

Execution flow:


```
Target method
↓
Exception escapes the method
↓
@AfterThrowing executes
↓
Exception continues toward the caller
```

21. Accessing the Exception

Use the `throwing` attribute to bind the exception to an advice parameter.

```
@AfterThrowing(
    pointcut =
        "execution(* " +
        "com. .studentmanagement.service.PaymentServi
ce.*(..))",
    throwing = "exception"
)
public void logException(Throwable exception) {

    System.out.println(
        "Exception type: " +
        exception.getClass().getSimpleName()
    );

    System.out.println(
        "Exception message: " +
        exception.getMessage()
    );
}
```

The `throwing` value must match the parameter name:

```
throwing = "exception"
```

```
public void logException(Throwable exception)
```

22. Restricting Advice to a Specific Exce

The parameter type can restrict when the advice executes.

```
@AfterThrowing(  
    pointcut =  
        "execution(* " +  
        "com. .studentmanagement.service.PaymentServi  
ce.*(..))",  
    throwing = "exception"  
)  
public void handleInvalidAmount(  
    IllegalArgumentException exception  
) {  
    System.out.println(  
        "Invalid payment request: " +  
        exception.getMessage()  
    );  
}
```

This advice executes for:

- `IllegalArgumentException`
- Compatible subclasses

It does not execute for unrelated exception types.

23. Does `@AfterThrowing` Handle the Exception?

Not automatically.

It observes the failed exit, but the original exception still propagates.

```
Target throws exception
    ↓
@AfterThrowing logs it
    ↓
Original exception continues
    ↓
Controller or caller receives it
```

The advice may itself throw another exception, but merely returning from an `@AfterThrowing` method does not consume the original failure.

24. Can `@AfterThrowing` Return a Fallback Value?

No.

```
@AfterThrowing("...")
public String returnFallback(Exception exception) {
    return "Fallback";
}
```

The target method has already completed exceptionally. The advice method's return value is not used as a recovery result.

Use `@Around` when the invocation must be caught and converted into a fallback value. This must be done carefully because silently swallowing failures can create incorrect application behaviour.

25. Suitable Uses of `@AfterThrowing`

Use it for behaviour that applies specifically to failed method execution:

- Error logging
- Failure metrics
- Failed-operation audit records
- Alert creation
- Recording exception type and message

- Technical cleanup that applies only to failure
-

@After Advice

26. @After Means After-Finally

The name can be misleading. `@After` does not mean “after successful completion.”

It behaves like a `finally` block:

```
try {  
    targetMethod();  
} finally {  
    afterAdvice();  
}
```

It executes whether the target method:

- Returns successfully
- Throws an exception

Spring therefore describes it as **after-finally advice**.

27. @After Example

```
@After(  
    "execution(* " +  
    "com. [redacted] .studentmanagement.service.PaymentService.*  
    (...))"  
)  
public void clearContext() {  
    System.out.println("Clearing method context");  
}
```

Execution on success:

```
Target returns normally
↓
@After executes
```

Execution on failure:

```
Target throws an exception
↓
@After executes
↓
Exception continues
```

28. Comparing the Three After Advices

Advice	Success	Failure	Direct access to result	Direct access to exception
<code>@AfterReturning</code>	Yes	No	Yes	No
<code>@AfterThrowing</code>	No	Yes	No	Yes
<code>@After</code>	Yes	Yes	No	No

`@After` vs `@AfterReturning`

```
@After          → success or failure
@AfterReturning → success only
```

`@After` vs `@AfterThrowing`

```
@After          → every outcome
@AfterThrowing → failure only
```

29. Suitable Uses of `@After`

Use `@After` for logic that must happen regardless of outcome:

- Releasing a lock
- Clearing contextual data
- Removing a `ThreadLocal` value
- Releasing a technical resource
- Generic method-exit logging

Use `@AfterReturning` when the returned value is needed.

Use `@AfterThrowing` when the thrown exception is needed.

`@After` is intentionally outcome-neutral.

`@Around` Advice

30. Why Is `@Around` the Most Powerful Advice?

`@Around` surrounds the complete invocation.

```
Code before
  ↓
Target method
  ↓
Code after
```

Unlike the other advice types, it controls the target method's execution.

It can:

- Run logic before the target method
- Run logic after the target method
- Inspect and replace arguments
- Inspect and replace the returned value
- Catch, rethrow, wrap, or replace exceptions
- Skip the target method entirely
- Invoke the method more than once

- Measure execution duration

Because it has this level of control, it should be used only when actually required.

31. Basic `@Around` Syntax

```
@Around("...")
public Object logAround(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    System.out.println("Before target method");

    Object result = joinPoint.proceed();

    System.out.println("After target method");

    return result;
}
```

Execution flow:

```
Enter @Around advice
    ↓
Execute code before proceed()
    ↓
Call proceed()
    ↓
Remaining advice and target execute
    ↓
Target result returns to the advice
    ↓
Execute code after proceed()
    ↓
Return result to caller
```

32. What Is `ProceedingJoinPoint` ?

`ProceedingJoinPoint` represents the current intercepted invocation ability to continue that invocation.

It is used specifically with `@Around` advice.

`ProceedingJoinPoint` extends `JoinPoint`, so it provides the norm information plus control methods such as:

```
proceed();  
proceed(Object[] arguments);
```

The core distinction is:

```
JoinPoint  
→ inspect the current invocation  
  
ProceedingJoinPoint  
→ inspect and control the current invocation
```

33. What Does `proceed()` Actually Do?

```
Object result = joinPoint.proceed();
```

This means:

Continue with the remaining interceptor chain and eventually invoke the actual target method.

It is not always equivalent to directly calling the business method because other interceptors or advice may still remain inside the chain.

```
First around advice  
↓  
proceed()  
↓  
Second advice  
↓
```



```
Transaction interceptor
    ↓
Security interceptor
    ↓
Actual target method
```

Therefore, `proceed()` means:

| Continue to the next step in the current invocation chain.

34. What Happens When `proceed()` Is Not Called?

```
@Around("...")
public Object blockExecution(
    ProceedingJoinPoint joinPoint
) {
    System.out.println("Method blocked");
    return "Blocked response";
}
```

The target method never executes.

```
Caller
    ↓
@Around advice
    ↓
No proceed()
    ↓
Target method skipped
    ↓
Alternative value returned
```

This ability can support patterns such as:

- Caching
- Feature flags
- Access rejection

- Short-circuiting
- Idempotency
- Circuit-breaking

It can also make behaviour difficult to understand because it appears to call a method that an aspect silently prevents from

35. Why Must We Return the Result of `proceed()` ?

Consider this incorrect advice:

```
@Around("...")
public Object incorrectAdvice(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    joinPoint.proceed();
    return null;
}
```

Even when the target returns:

Payment successful

the caller receives:

null

The value returned by the `@Around` advice becomes the value observed by the caller.

Therefore, the normal pattern is:

```
Object result = joinPoint.proceed();
```

```
return result;
```

Return something else only when deliberately replacing the t

36. Why Does Around Advice Usually Re

A generic aspect may intercept methods with different return

```
String  
Student  
PaymentReceipt  
List<Order>  
int  
boolean  
void
```

Therefore, a reusable around advice normally uses:

```
public Object advice(...) throws Throwable
```

Primitive values are boxed:

```
int      → Integer  
boolean → Boolean
```

For a `void` target method, the result is effectively `null`.

Declaring a generic around advice as `void` would discard any value returned by `proceed()` and cause the caller to observe `null` where a value was expected.

Practical Uses of `@Around`

37. Measuring Execution Time

Execution-time measurement requires state that exists both before and after the same method invocation:

```
Start time before method
End time after method
```

This makes `@Around` a natural choice.

```
@Around(
    "execution(* " +
    "com. .... .studentmanagement.service.PaymentService.*"
    " ..)"
)
public Object measureExecutionTime(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    long startTime = System.nanoTime();

    try {
        return joinPoint.proceed();
    } finally {

        long executionTime =
            System.nanoTime() - startTime;

        System.out.println(
            joinPoint.getSignature().getName() +
            " took " +
            executionTime +
            " nanoseconds"
        );
    }
}
```

Why use `finally`?

This version is incomplete:

```

long startTime = System.nanoTime();

Object result = joinPoint.proceed();

long executionTime =
    System.nanoTime() - startTime;

```

If `proceed()` throws an exception, execution never reaches the duration calculation. Using `finally` ensures that timing is recorded on both success and failure.

38. Exception Behaviour Inside `@Around`

```

@Around("...")
public Object observeExecution(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    System.out.println("Before");

    Object result = joinPoint.proceed();

    System.out.println("After");

    return result;
}

```

When the target throws an exception:

```

"Before" is printed
proceed() throws
"After" is not printed
Exception propagates

```

`proceed()` behaves like an ordinary Java method call. If it throws an exception, it does not run unless the exception is caught or a `finally` block is present.

39. Observing Success and Failure in On

```
@Around("...")
public Object observeExecution(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    System.out.println(
        "Starting: " +
        joinPoint.getSignature().getName()
    );

    try {
        Object result = joinPoint.proceed();

        System.out.println("Execution successful");
        return result;
    } catch (Throwable exception) {

        System.out.println(
            "Execution failed: " +
            exception.getMessage()
        );

        throw exception;
    } finally {
        System.out.println("Execution completed");
    }
}
```

This single advice can reproduce the major lifecycle position

- `@Before`
- `@AfterReturning`
- `@AfterThrowing`
- `@After`

That does not mean `@Around` should always be used. When only one lifecycle position is required, a narrower advice communicates the intention more clearly.

40. Converting an Exception into a Fallback Value

```
@Around("...")
public Object provideFallback(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    try {
        return joinPoint.proceed();
    } catch (PaymentGatewayException exception) {

        return new PaymentReceipt(
            "PENDING",
            "Payment will be retried"
        );
    }
}
```

Here, the original exception does not reach the caller. The aspect converts a failed invocation into a successful return value.

This is powerful but potentially dangerous:

- Monitoring may no longer detect the original failure.
- Transaction behaviour may change.

- Callers may believe the operation succeeded.
- Business failure rules become hidden inside infrastructure

Fallback, retry, and circuit-breaker policies are usually safer through established resilience infrastructure rather than an in aspect.

41. Modifying Method Arguments

Suppose the target method is:

```
public String greetStudent(String name) {  
    return "Hello " + name;  
}
```

Around advice can replace the argument before continuing the invocation:

```
@Around(  
    "execution(* " +  
    "com._____.studentmanagement.service.StudentService"  
    +  
    ".greetStudent(..))"  
)  
public Object normalizeName(  
    ProceedingJoinPoint joinPoint  
) throws Throwable {  
  
    Object[] arguments = joinPoint.getArgs();  
  
    String originalName =  
        (String) arguments[0];  
  
    String normalizedName =  
        originalName.trim().toUpperCase();  
  
    Object[] modifiedArguments = {
```



```

        normalizedName
    };

    return joinPoint.proceed(modifiedArguments
}

```

Calling:

```

studentService.greetStudent(" aditya ");

```

causes the target method to receive:

```

ADITYA

```

The new argument array must remain compatible with the target method's parameter types and order.

42. Modifying the Returned Value

```

@Around(
    "execution(* " +
    "com. .studentmanagement.service.PaymentService"
+
    ".makePayment(..))"
)
public Object wrapResponse(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    Object originalResult =
        joinPoint.proceed();

    return new ApiResponse<>(
        true,

```

```

        "Operation completed",
        originalResult
    );
}

```

The caller receives the wrapper instead of the target method

The replacement value must be compatible with the target method return type. Otherwise, the caller may receive a runtime type error.

For example, a method declared to return `String` cannot safely be replaced with an unrelated `ApiResponse` object unless the original method signature is designed to return that type.

43. Calling `proceed()` More Than Once

Around advice may call `proceed()`:

- Once
- More than once
- Not at all

Example retry:

```

@Around("...")
public Object retry(
    ProceedingJoinPoint joinPoint
) throws Throwable {

    try {
        return joinPoint.proceed();
    } catch (TemporaryServiceException exception) {
        return joinPoint.proceed();
    }
}

```

This may invoke the target method twice.

That is dangerous for operations with side effects:

```
transferMoney()  
sendEmail()  
createOrder()  
chargeCard()
```

Calling such methods twice may create duplicate operations.

Multiple calls to `proceed()` should be considered only when:

- The operation is safe to retry.
- Idempotency is guaranteed.
- The retry policy is clearly defined.
- Failure categories are carefully selected.

For production applications, specialized retry infrastructure is generally safer than a simplistic custom retry aspect.

Choosing the Correct Advice Type

44. Advice Comparison

Requirement	Best advice type
Run logic before a method	<code>@Before</code>
Observe a successful result	<code>@AfterReturning</code>
Observe a failure and access the exception	<code>@AfterThrowing</code>
Run cleanup regardless of outcome	<code>@After</code>
Measure complete execution time	<code>@Around</code>
Replace method arguments	<code>@Around</code>
Replace or wrap the return value	<code>@Around</code>
Skip the method and return another value	<code>@Around</code>

Requirement	Best advice type
Catch an exception and return a fallback	@Around
Retry the invocation	@Around

45. Prefer the Narrowest Suitable Advice

The most powerful tool is not always the clearest tool.

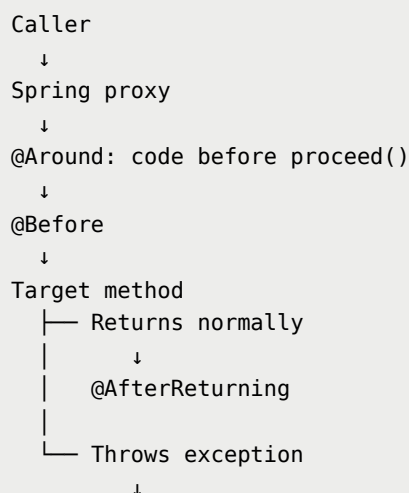
Use:

- @Before when only pre-execution logic is required.
- @AfterReturning when only successful completion matters.
- @AfterThrowing when only failed completion matters.
- @After when cleanup must happen for every outcome.
- @Around only when the invocation itself must be controlled.

A narrow advice makes the lifecycle position obvious and reduces the chance of accidentally changing business behaviour.

Final Mental Model

46. Complete Invocation Flow



@AfterThrowing

↓

@After executes in either case

↓

Control returns to @Around

↓

@Around returns a result or propagates an exception

↓

Caller

The exact ordering becomes more nuanced when several aspects and interceptors are involved, but this flow provides the correct conceptual foundation for understanding the role of each advice type.

Key Takeaways

1. Advice is the additional behaviour applied to matched method executions.
2. A pointcut expression decides which methods are matched.
3. Spring normally exposes a proxy in place of the original target object.
4. `JoinPoint` allows an advice to inspect the current invocation.
5. `ProceedingJoinPoint` allows `@Around` advice to inspect and control the invocation.
6. `@Before` runs before the target but cannot directly replace arguments or return an alternate result.
7. `@AfterReturning` runs only after normal completion and cannot replace the returned value.
8. `@AfterThrowing` runs only when an exception escapes the target and does not automatically consume it.
9. `@After` behaves like `finally` and runs on both success and failure.
10. `@Around` can modify, skip, repeat, or replace the invocation, so it must be used carefully.
11. `proceed()` continues the remaining interceptor chain and eventually reaches the target method.

12. Prefer the narrowest advice type that correctly expresses